Min-Cut Algorithms Overview

L10_S1.pdf
PDF

Generate a lecture scribe for exam preparation based on the provided PDF covering the 'Min-Cut Problem, Max-Flow Min-Cut Theorem, Stoer-Wagner Deterministic Algorithm, and Karger's Randomized Algorithm'. Reason step-by-step through definitions, edge contraction assumptions, and example solutions exactly as presented. Explicitly derive the logical flow for successful versus unsuccessful runs and algorithm comparisons in paragraph format. Do not introduce external material or derivations not found in the context.

Lecture 10 Scribe – Min-Cut Algorithms and Theory (Exam Preparation Notes)
(Based strictly on the provided lecture PDF; step-by-step reasoning and logical flow preserved)

---

1. Min-Cut Problem: Motivation and Applications

The min-cut problem arises in situations where we want to understand how strongly connected a network is and how easily it can be separated. The lecture motivates its importance through applications centered on connectivity, reliability, and optimization.

The algorithm is used in:

- **Network design:** It helps improve communication efficiency and optimize network flow by identifying the minimum capacity cut.
- **Communication networks:** It helps analyze vulnerability to failures and supports building fault-tolerant systems.
- **VLSI design:** It helps partition circuits into smaller components and reduce interconnect complexity.

Thus, the core purpose of min-cut is to identify the smallest set of edges whose removal disrupts connectivity in a meaningful way.

2. Definition of Cut and Min-Cut

A **cut-set** in a graph is defined as a set of edges whose removal breaks the graph into two or more connected components.

Given a graph $G = (V, E)$ with n vertices, the **minimum cut (min-cut)** problem is to find a cut-set with the smallest possible number of edges.

This definition establishes the optimization objective: find the cut with minimum cardinality.

---

3. Edge Contraction: Core Assumption of Min-Cut Algorithms

The fundamental operation in many min-cut algorithms is **edge contraction**.

Step-by-step reasoning of contraction:

1. Select an edge (u, v).
2. Remove that edge.
3. Merge the two vertices u and v into a single vertex.
4. Remove any edges that directly connected u and v.
5. Keep all other edges.

After contraction:

- The new graph may contain **parallel edges**.
- It will **not contain self-loops**.

This operation reduces the number of vertices while preserving connectivity structure relevant to cuts.

---

4. Logical Flow: Successful vs Unsuccessful Min-Cut Runs

Successful Run

A **successful min-cut run** occurs when the algorithm correctly identifies the true minimum cut.

The reasoning is:

- The algorithm repeatedly contracts edges.

- If it avoids contracting edges that belong to the true minimum cut, then the structure of that cut survives until the end.
- Eventually, the graph is reduced while preserving the critical cut, and the algorithm outputs the correct minimum cut.

Thus, success depends on not destroying important cut edges during contraction.

Unsuccessful Run

An **unsuccessful min-cut run** happens when the algorithm fails to find the correct minimum cut.

Logical reasoning:

- During contraction, if a critical edge from the true minimum cut is selected and contracted early, the structure of that cut is destroyed.
- Once destroyed, the algorithm can no longer recover the correct partition.
- The final result corresponds to a larger cut, not the minimum one.

Hence, the correctness of the result depends on the sequence of contractions. This explains why randomized approaches may succeed or fail depending on edge choices.

---

5. Max-Flow Min-Cut Theorem

The theorem states:

> In a flow network, the maximum amount of flow from the source to the sink equals the total weight of the edges in a minimum cut.

To understand this logically, define the following:

- **Capacity of a cut:** Sum of capacities of edges going from vertices in set X to vertices in set Y.
- **Minimum cut:** The cut with the smallest capacity.
- **Maximum flow:** The largest possible flow from source S to sink T.

The theorem connects two quantities:

- The best possible flow achievable.
- The smallest capacity barrier separating source and sink.

Thus, the flow is limited exactly by the weakest cut in the network.

---

6. Deterministic Min-Cut: Stoer–Wagner Algorithm

The Stoer–Wagner algorithm provides an exact solution using a deterministic process.

Core Logical Principle

Let s and t be vertices of graph G. Let $G/\{s, t\}$ be the graph obtained by merging them.

Then:

A minimum cut of G is the smaller of:

- A minimum s–t cut of G, or
- A minimum cut of the merged graph $G/\{s, t\}$.

Reasoning:

- Case 1: If a true minimum cut separates s and t, then the minimum s–t cut already represents the global minimum cut.
- Case 2: If no minimum cut separates s and t, then merging them does not destroy the true minimum cut. Therefore, the minimum cut exists in the contracted graph.

This gives a recursive logic: evaluate cuts while gradually shrinking the graph.

Algorithm Structure

MinimumCutPhase(G, a):

- Start with a set $A = \{a\}$.
- Repeatedly add the vertex most tightly connected to A.
- When all vertices are added, the final added pair defines a cut whose weight is recorded.

MinimumCut(G):

1. Repeat while vertices remain.
2. Run a phase.
3. If the phase cut is lighter than current best, store it.
4. Shrink the graph by merging the last two added vertices.
5. Continue until completion.

This process ensures an exact minimum cut is found.

---

7. Randomized Min-Cut: Motivation

Randomized algorithms are used because they:

- Provide probabilistic guarantees of success.
- Can estimate the minimum cut using fewer iterations.
- Improve efficiency and enable parallelization.
- Avoid worst-case behavior.
- Offer robustness and heuristic flexibility.

---

8. Karger's Randomized Algorithm

Karger's algorithm repeatedly contracts randomly chosen edges until only two vertices remain. The remaining edges between the two supernodes define a cut.

Key reasoning:

- If random contraction avoids destroying true min-cut edges, the final cut equals the minimum cut.
- If a min-cut edge is contracted early, the run becomes unsuccessful.

Thus, correctness is probabilistic rather than guaranteed.

---

9. Probability of Finding the Min-Cut

The lecture states that the algorithm outputs a minimum cut set with probability at least:

$$\frac{2}{n(n-1)}$$

This shows that one run has limited success probability, which motivates repeating the algorithm multiple times to improve reliability.

---

10. Deterministic vs Randomized Min-Cut: Comparison

The choice between methods depends on the problem context.

Deterministic (Stoer–Wagner)

- Always guarantees the exact minimum cut.

- May have higher time complexity for large graphs.
- Time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized (Karger)

- Produces the minimum cut with high probability.
- Time complexity:

$$O(V^2)$$

Thus, the deterministic method provides certainty, while the randomized method trades certainty for speed and simplicity.

---

11. Logical Summary for Exam Understanding

The min-cut problem is defined as finding the smallest set of edges whose removal disconnects a graph. The central operation in many solutions is edge contraction, which reduces the graph while preserving cut structures.

A successful run occurs when important cut edges are never contracted; an unsuccessful run occurs when they are contracted early. This distinction explains why randomized algorithms may fail on some runs but succeed on others.

The Max-Flow Min-Cut theorem provides a theoretical foundation linking flows and cuts. Deterministic methods like Stoer–Wagner guarantee exact answers through structured merging and evaluation of cuts. Randomized methods like Karger's rely on probabilistic success through random edge contractions and repeated trials.

The comparison ultimately highlights a trade-off:

- Deterministic → guaranteed correctness, higher complexity.
- Randomized → faster and simpler, probabilistic correctness.

L10_S1.pdf
PDF

Generate a lecture scribe for exam preparation based on the provided PDF covering the 'Min-Cut Problem, Max-Flow Min-Cut Theorem, Stoer-Wagner Deterministic Algorithm, and Karger's Randomized Algorithm'. Reason step-by-step through definitions, edge contraction assumptions, and example solutions exactly as presented. Explicitly derive the logical flow for successful versus unsuccessful runs and algorithm comparisons in paragraph format. Do not introduce external material or derivations not found in the context.

Lecture 10 Scribe – Min-Cut Problem, Max-Flow Min-Cut Theorem, Stoer–Wagner Algorithm, and Karger's Randomized Algorithm (Exam Preparation Notes)
(Strictly based on the provided lecture PDF; step-by-step logical flow, definitions, and reasoning preserved. Paragraph format as requested.)

---

1. Understanding the Min-Cut Problem

The min-cut problem is motivated by the need to analyze connectivity, reliability, and optimization in networks. The lecture explains that the algorithm is used in applications such as network design, communication networks, and VLSI design, where identifying the smallest separating structure helps improve efficiency, robustness, and partitioning. Thus, the core purpose is to identify how a network can be separated using the smallest possible number of edges.

A cut-set in a graph is defined as a set of edges whose removal breaks the graph into two or more connected components. Given a graph $G = (V, E)$ with n vertices, the min-cut problem asks for a cut-set with the minimum possible cardinality. The objective is therefore to find the smallest number of edges whose removal disconnects the graph.

A central assumption in min-cut algorithms is the operation of edge contraction. Edge contraction removes an edge while merging its two endpoints into a single vertex. During contraction of an edge (u, v), the vertices u and v are merged, all edges connecting them are eliminated, and all other edges are retained. The

resulting graph may contain parallel edges but does not contain self-loops. This operation reduces the graph while preserving structural information about cuts.

---

2. Logical Flow of Successful vs Unsuccessful Min-Cut Runs

A successful min-cut run refers to an iteration of an algorithm that correctly identifies the minimum cut of a graph. The logic behind this success is rooted in the contraction process. If the algorithm contracts edges that are not part of the true minimum cut, then the structure of the minimum cut remains preserved throughout the reduction of the graph. As contraction continues, the graph shrinks while maintaining the critical separation represented by the true minimum cut. Eventually, the final partition corresponds to the correct minimum cut, making the run successful.

An unsuccessful min-cut run occurs when the algorithm fails to identify the correct minimum cut. The reasoning follows from the same contraction assumption. If an edge belonging to the true minimum cut is contracted early in the process, then the separation that defines the minimum cut is destroyed. Once that structural boundary is merged, the algorithm can no longer recover the correct cut. As a result, the final partition corresponds to a larger or incorrect cut, and the run is unsuccessful. This explains why algorithms sensitive to edge selection can produce different outcomes across runs.

---

3. Max-Flow Min-Cut Theorem

The Max-Flow Min-Cut theorem provides a theoretical foundation connecting flows and cuts. It states that in a flow network, the maximum amount of flow that can pass from the source to the sink is equal to the total weight of the edges in a minimum cut.

To understand this relationship step-by-step, the lecture defines the following components. The capacity of a cut is the sum of capacities of edges that go from vertices in set X to vertices in set Y. The minimum cut is the cut with the smallest capacity among all possible cuts. The minimum cut capacity is simply the capacity value of that cut. The maximum flow is the largest amount of flow that can be sent from source S to sink T. The theorem then establishes that the value of the

maximum flow equals the capacity of the minimum cut, showing that the weakest separation in the network determines the maximum possible flow.

---

4. Stoer–Wagner Deterministic Min-Cut Algorithm

The Stoer–Wagner algorithm is a deterministic method that guarantees finding the exact minimum cut. The logical principle behind it is based on merging vertices and comparing cuts before and after merging.

Let s and t be two vertices of graph G, and let $G/\{s, t\}$ denote the graph obtained by merging them. A minimum cut of G can be obtained by taking the smaller of two possibilities: the minimum s–t cut of G, or the minimum cut of the merged graph $G/\{s, t\}$.

The reasoning is as follows. Either a minimum cut exists that separates s and t, in which case the minimum s–t cut already represents the true minimum cut of the graph. Otherwise, there is no minimum cut separating them, meaning merging s and t does not destroy the minimum cut. In that case, the minimum cut must exist in the contracted graph. This step-by-step logic ensures correctness across recursive contractions.

The algorithm proceeds in phases. Starting from a chosen vertex a, a set A is formed. Vertices are added one by one to A based on tight connectivity until all vertices are included. The weight of the cut defined at the end of this phase is recorded as the "cut of the phase." If this cut is smaller than the current minimum, it is stored. Then the graph is shrunk by merging the last two added vertices, and the process repeats. Eventually, the smallest cut observed is returned as the minimum cut.

---

5. Karger's Randomized Min-Cut Algorithm

Randomized algorithms are introduced because they provide probabilistic guarantees of success and can estimate the minimum cut using fewer iterations. The lecture notes emphasize benefits such as efficiency, parallelization, approximation guarantees, avoidance of worst-case instances, heuristic nature, and robustness.

Karger's algorithm is based on repeated random edge contraction. At each step, an edge is chosen randomly and contracted. This continues until only two super-vertices

remain. The edges between these two remaining vertices form a cut. If during the contraction process the algorithm avoids contracting edges from the true minimum cut, the final result corresponds to the correct minimum cut. If such edges are contracted early, the final cut will be incorrect. Thus, success depends on the random choices made during contraction.

The lecture states that the algorithm outputs a minimum cut set with probability at least $\frac{2}{n(n-1)}$, where n is the number of vertices. This shows that a single run may fail, but repeated runs increase the chance of finding the true minimum cut.

---

6. Comparison: Deterministic vs Randomized Min-Cut

The choice between deterministic and randomized approaches depends on the specific problem context. The deterministic Stoer–Wagner algorithm always guarantees an exact minimum cut. However, it may have higher time complexity for large graphs, with complexity given as $O(V \cdot E + V^2 \log V)$.

In contrast, the randomized approach using Karger's algorithm provides an approximate minimum cut with high probability. Its time complexity is $O(V^2)$, making it simpler and faster, but without certainty in a single run.

The logical trade-off is therefore clear. Deterministic algorithms rely on structured vertex merging and cut evaluation to guarantee correctness, while randomized algorithms rely on repeated trials and probabilistic success, where the outcome depends on whether critical edges are preserved during contractions.

---

7. Conceptual Summary for Exam Revision

The min-cut problem seeks the smallest set of edges that disconnects a graph. Edge contraction is the core operation that simplifies the graph while preserving structural information about cuts. A successful run occurs when critical edges are preserved; an unsuccessful run occurs when they are contracted prematurely. The Max-Flow Min-Cut theorem provides the theoretical bridge between network flow and minimum separation. The Stoer–Wagner algorithm deterministically finds the exact minimum cut using structured merging and comparison logic, while Karger's randomized algorithm uses repeated random contractions and achieves the correct result with a known probability.

L10_S1.pdf
PDF

Generate a lecture scribe for exam preparation based on the provided PDF covering the 'Min-Cut Problem, Max-Flow Min-Cut Theorem, Stoer-Wagner Deterministic Algorithm, and Karger's Randomized Algorithm'. Reason step-by-step through definitions, edge contraction assumptions, and example solutions exactly as presented. Explicitly derive the logical flow for successful versus unsuccessful runs and algorithm comparisons in paragraph format. Do not introduce external material or derivations not found in the context.

Lecture 10 Scribe – Exam Preparation Notes
Topics Covered: Min-Cut Problem, Max-Flow Min-Cut Theorem, Stoer–Wagner Deterministic Algorithm, Karger's Randomized Algorithm
(Strictly derived from the provided PDF. Step-by-step reasoning preserved. Paragraph format. No external additions.)

Min-Cut Problem: Definition, Purpose, and Edge-Contraction Assumptions

The min-cut problem is studied in the context of understanding connectivity, reliability, and optimization in networks. The lecture explains that min-cut is used in applications such as network design, communication networks, and VLSI design, where identifying the smallest separating structure helps improve efficiency and robustness. The main goal is to determine how a graph can be disconnected by removing the fewest edges.

A cut-set in a graph is defined as a set of edges whose removal breaks the graph into two or more connected components. Given a graph $G = (V, E)$ with n vertices, the minimum cut (min-cut) problem is to find the cut-set having the smallest possible

number of edges. This establishes the optimization objective: find the minimum cardinality set of edges that disconnects the graph.

The main operation underlying min-cut algorithms is edge contraction. This operation removes an edge and merges its two endpoint vertices into a single vertex. During contraction of an edge (u, v), the two vertices are combined, all edges between them are removed, and all other edges remain. The resulting graph may contain parallel edges but does not contain self-loops. This assumption is crucial because the algorithm repeatedly simplifies the graph while preserving the structure needed to identify cuts.

Min-cut algorithms such as Karger's algorithm are random and sensitive to the choice of edges. If critical edges are contracted early, the true minimum cut may be lost. This observation forms the basis for understanding success and failure in randomized runs.

Logical Flow of Successful vs Unsuccessful Runs

A successful min-cut run refers to the correct outcome of an algorithm that identifies the true minimum cut. The logic behind a successful run follows from the contraction process. As the algorithm repeatedly contracts edges, if it avoids contracting edges that belong to the true minimum cut, then the structural separation corresponding to the minimum cut remains intact. Eventually, when the graph is reduced to a simpler structure, the preserved separation is revealed as the minimum cut. Thus, success depends on preserving critical edges throughout the contraction process.

An unsuccessful min-cut run occurs when the algorithm fails to identify the true minimum cut. The reason for failure is again tied to contraction. If an edge that belongs to the actual minimum cut is selected and contracted early, then the boundary defining the minimum cut is destroyed. Once this happens, the algorithm cannot recover the correct separation, and the final cut identified will be larger or incorrect. Therefore, the success or failure of a run is determined by whether essential cut edges are preserved or destroyed during the contraction steps.

Max-Flow Min-Cut Theorem: Step-by-Step Conceptual Flow

The Max-Flow Min-Cut theorem states that in a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut. This theorem establishes a direct equality between two quantities: maximum achievable flow and minimum separating capacity.

To understand this, the lecture defines key terms. The capacity of a cut is the sum of capacities of edges oriented from vertices in set X to vertices in set Y. The minimum cut is the cut having the smallest capacity among all possible cuts, and its capacity is called the minimum cut capacity. The maximum flow is the largest possible flow from source S to sink T. The theorem logically connects these ideas by stating that the smallest barrier separating the source and sink limits the maximum possible flow, and these two quantities are equal.

Stoer–Wagner Deterministic Min-Cut Algorithm: Logical Derivation

The Stoer–Wagner algorithm is a deterministic method that always guarantees the exact minimum cut. Its reasoning is based on repeatedly merging vertices and comparing possible cuts.

Let s and t be two vertices in a graph G, and let $G/\{s, t\}$ be the graph obtained by merging them. The minimum cut of G can be obtained by taking the smaller of the following two values: the minimum s-t cut of G, or the minimum cut of the merged graph $G/\{s, t\}$.

The logic behind this theorem proceeds in two cases. If a minimum cut exists that separates s and t, then the minimum s-t cut is itself the global minimum cut. Otherwise, if no minimum cut separates s and t, then merging them does not remove the true minimum cut, and the minimum cut must be found in the contracted graph. This reasoning ensures correctness through repeated merging.

The algorithm proceeds in phases. Starting with a vertex a, a set A is formed. Vertices are repeatedly added to A based on which vertex is most tightly connected to it. Once all vertices are included, the weight of the resulting separation is taken as the “cut of the phase.” If this cut is lighter than the best cut found so far, it becomes the current minimum. Then the graph is shrunk by merging the last two added vertices, and the process repeats. At the end, the smallest cut observed across phases is returned as the minimum cut.

Karger's Randomized Algorithm: Step-by-Step Logical Flow

Randomized algorithms are introduced because they provide probabilistic guarantees of success and can estimate the minimum cut using fewer iterations. The lecture emphasizes properties such as efficiency, parallelization, approximation guarantees, avoidance of worst-case instances, heuristic flexibility, and robustness.

Karger's randomized algorithm works by repeatedly contracting randomly selected edges. Each contraction reduces the number of vertices while preserving possible cut structures. If the process continues without contracting any edge belonging to the true minimum cut, then the final partition obtained when only two super-vertices remain corresponds to the correct minimum cut. However, if at any stage an edge from the true minimum cut is contracted, then the minimum cut structure is destroyed, leading to an incorrect result. Thus, the algorithm's correctness depends on random edge choices.

The lecture states that the algorithm outputs a minimum cut set with probability at least $2/(n(n-1))$, which shows that a single run may not always succeed, but repeated runs improve the chances of obtaining the correct cut.

Algorithm Comparison: Deterministic vs Randomized

The comparison between deterministic and randomized approaches is based on accuracy and complexity. The deterministic Stoer–Wagner algorithm always guarantees an exact minimum cut. However, it may have higher time complexity for large graphs, with complexity given as $O(V \cdot E + V^2 \log V)$.

In contrast, the randomized approach using Karger's algorithm provides an approximate minimum cut with high probability. Its time complexity is $O(V^2)$, making it computationally simpler. However, unlike the deterministic method, it does not guarantee correctness in a single run, since success depends on not contracting critical edges.

Thus, the logical distinction is clear. The deterministic method relies on structured merging and guaranteed correctness, while the randomized method relies on repeated probabilistic trials where success depends on preserving the true cut during contractions.